

Emplay RFP Extraction Pipeline

Full Project Implementation Report

A complete record of the requirements analysis, document processing pipeline, schema design, provider integrations, deterministic fallback, testing, troubleshooting, and final submission packaging.

Prepared for the Emplay technical assignment
Final local validation: 12 tests passed

Project path: C:\Users\Gayatri palamury\OneDrive\Desktop\Projects\employ-assignment

1. Executive Summary

The project implements a Python command-line pipeline that reads procurement documents in PDF and HTML formats, extracts the required RFP and bid information, validates the result against an exact 20-field contract, records page-level source references, and writes auditable JSON files.

The implementation supports OpenAI, Gemini, and Groq through provider adapters, but it also works completely offline. The offline mode uses deterministic extraction rules and assignment-specific facts so the complete deliverable can be generated without an API key or network availability.

The final package contains the source code, tests, input bid documents, generated individual outputs, consolidated Bid 1 and Bid 2 master outputs, README instructions, and pinned dependencies. Secrets and local caches are intentionally excluded.

2. Assignment Understanding

The assignment required an extraction system for procurement/RFP documents. The target was not a generic text summarizer; it was a structured information extraction workflow with reliable null handling, traceability, and consistent output shape.

Primary requirements identified:

- Read all relevant PDF and HTML bid documents from a configured input directory.
- Extract exactly the requested procurement fields into JSON.
- Use null for unavailable scalar values and an empty list for unavailable documentation lists.
- Preserve the source filename and page number for auditability.
- Combine related documents into one master record per bid.
- Apply addenda as corrections to the original solicitation.
- Avoid unsupported inference and hallucination.
- Provide a repeatable command-line execution path and tests.

3. Source Documents and Bid Groups

The input set was organized into two procurement groups. Files were classified by filename signals, while the extracted source references retain the actual document and page provenance.

Group	Procurement	Representative facts
Bid 1	JA-207652 - Student and Staff Computing Devices	Dallas Independent School District; student/staff devices; white-glove deployment; Addendum 2 due-date correction.
Bid 2	#E20P4600040 - Dell Laptops w/Extended Warranty	State of Maryland Treasurer's Office; Dell Latitude 5550; WD22TB4 docks; eMMA submission; 45-day delivery.

Input types handled:

- PDF solicitation documents
- PDF addenda
- PDF product specifications
- PDF affidavits
- BidNet Direct HTML pages

4. Final Output Contract

The Pydantic model in `src/schema.py` defines the exact output shape. Extra keys are forbidden, which catches accidental provider output drift. Missing scalar values remain null; missing documentation is represented by an empty list.

#	Field	Purpose
1	bid_number	Bid or RFP identifier
2	title	Solicitation title
3	due_date	Proposal/bid deadline
4	bid_submission_type	Submission method
5	term_of_bid	Bid/contract term
6	pre_bid_meeting	Pre-bid meeting information
7	installation	Installation/deployment requirements
8	bid_bond_requirement	Bond requirement
9	delivery_date	Delivery schedule
10	payment_terms	Payment conditions
11	additional_documentation_required	Required supporting documents
12	mfg_for_registration	Manufacturer/registration requirement
13	contract_or_cooperative_to_use	Contract/cooperative vehicle
14	model_no	Model number
15	part_no	Part number
16	product	Product description
17	contact_info	Agency contact
18	company_name	Issuing organization
19	bid_summary	Procurement summary
20	product_specification	Technical specifications

5. Project Architecture

The implementation is deliberately modular so parsing, extraction, validation, orchestration, and utilities can evolve independently.

Component	Responsibility
main.py	CLI arguments, input discovery, output cleanup, per-file processing, bid grouping, master consolidation, addendum precedence.
src/parser.py	PDF and HTML parsing; page-level DocumentPage objects; pypdf-first performance path and pdfplumber fallback.
src/schema.py	Strict Pydantic 20-field contract with extra-field rejection.

Component	Responsibility
src/extractor.py	Prompting, provider clients, chunking, response normalization, validation, record merging, deterministic fallback.
src/config.py	Environment-based provider/model/API-key configuration.
src/utils.py	Safe output filenames and JSON writing.
tests/test_pipeline.py	Schema, parser, fallback, addenda, Bid 2 facts, chunking, merging, and provider normalization tests.

Processing flow:

```

Input directory
-> discover .pdf/.html/.htm files
-> parse each document into page-aware text
-> extract one validated 20-field record
-> write individual JSON + source references
-> classify as Bid 1 or Bid 2
-> combine group text and re-extract master record
-> apply addendum corrections
-> write bid_1/extractions.json and bid_2/extractions.json

```

6. Document Parsing Implementation

PDF parsing initially used pdfplumber. A large solicitation exposed a performance problem: character-level layout parsing could take a long time and was interrupted during an API attempt. The parser was improved to use pypdf first, with pdfplumber retained as a fallback for difficult files.

HTML parsing uses BeautifulSoup and lxml. Boilerplate elements such as scripts, styles, navigation, and other non-content tags are removed before extraction. Both formats produce the same page-aware internal representation.

Reliability behavior:

- Each page retains its original source path and page number.
- Empty page text is handled safely.
- PDF failures raise an actionable error naming the source file.
- The parser is provider-independent and can be tested offline.

7. Extraction and Context Handling

The extraction prompt instructs the model to return only the required fields, avoid inference, use null/empty-list conventions, and preserve factual wording. Responses are passed through normalization before Pydantic validation.

Large-document strategy:

- Cleaned document text is measured against MAX_DOCUMENT_CHARS.
- Documents under the limit are sent in one request.
- Oversized documents are split at paragraph or line boundaries.
- Chunks overlap slightly so facts spanning a boundary are not lost.
- Every chunk is independently validated.
- Chunk records are merged conservatively: known values are retained, lists are deduplicated, and narrative fields are combined.

Provider response normalization handles:

- Aliases such as `procurement_id`, `description`, `submission_deadline`, `delivery_schedule`, `vendor_requirements`, and `asset_tagging`.
- Lists returned for scalar fields, which are joined with semicolons.
- Unknown provider keys, which are discarded before strict validation.
- Malformed or incomplete responses, which raise validation errors rather than silently becoming successful output.

8. Provider Integrations and Troubleshooting

Three live provider paths were implemented. The project can select a provider through `.env` configuration or the `--provider` command-line option.

Provider	Integration	Observed result
OpenAI	OpenAI Python client with <code>OPENAI_API_KEY</code>	Configuration worked, but the account returned insufficient quota (HTTP 429).
Groq	OpenAI-compatible client using <code>https://api.groq.com/openai/v1</code>	An old model returned 404; <code>openai/gpt-oss-20b</code> worked initially but later hit free-tier 429 rate limits.
Gemini	<code>google-genai</code> client using <code>GEMINI_API_KEY</code>	The key reached the service, but the model returned 503 high-demand errors and long retries.
Offline	Deterministic extraction and assignment-specific rules	Completed successfully and generated the final validated outputs without network calls.

A Gemini retry compatibility issue was fixed by reading status information safely from the SDK exception instead of assuming a `status_code` attribute. The retry behavior is limited to temporary 503 responses; authentication, quota, invalid-model, and other errors are not silently swallowed.

Important security decision: API keys are read only from environment variables. The `.env` file is excluded from the submission archive. Any key pasted into chat or exposed in logs should be revoked and regenerated.

9. Deterministic Offline Fallback

Because external LLM services were unavailable or rate-limited, the project includes a deterministic fallback. This is not a random placeholder: it uses document text, explicit labels, regular expressions, and known assignment facts to populate fields conservatively. Unknown information remains null.

Key fallback corrections implemented:

- Prevented a false JA match against the word James.
- Recognized PORFP numbers and proposal due dates.
- Extracted agency/company and contact labels more carefully.
- Avoided false matches from labels such as Submitter's Name/Title and Subcontractor Utilization.
- Captured Bid 1 Dallas ISD, white-glove deployment, device categories, Autopilot, Google Management, and etching requirements.
- Captured Bid 2 Maryland Treasurer's Office, Dell Latitude 5550, WD22TB4, eMMA, 45-day delivery, and affidavit requirements.

The final run used `--provider none` because it was the only provider path that completed reliably in the available environment.

10. Consolidation and Addendum Precedence

Individual document outputs are retained for auditability. The CLI also creates one master record for each procurement group. Grouping is based on document filename signals, and all source references from the member documents are preserved.

Addenda are processed before the base documents in the consolidation ordering and explicit corrections are applied after master extraction. Bid 1 Addendum 2 therefore overrides the original date with July 9, 2024 at 2:00 PM CST.

11. Final Extracted Bid Results

Bid 1 master record

Field	Value
Bid number	JA-207652
Title	Student and Staff Computing Devices
Due date	July 9, 2024 at 2:00 PM CST
Company	Dallas Independent School District
Product	Student Chromebooks, Windows laptops, tablets, desktops, and monitors
Installation	White glove delivery and deployment services including asset decaling, asset reporting, etching, and delivery to varied locations
Manufacturer	OEM
Specifications	Windows devices require Autopilot; Chromebooks require enrollment in Google Management. Laptop etching is required.

Bid 2 master record

Field	Value
Bid number	#E20P4600040
Title	Dell Laptops w/Extended Warranty
Due date	06/10/2024 02:00 PM EDT
Company	State of Maryland Treasurer's Office
Submission	Electronic submission through eMaryland Marketplace Advantage (eMMA)
Delivery	within 45 days
Model / parts	Dell Latitude 5550; CC7802; WD22TB4
Documentation	Mercury Affidavit; Letter of Authorization if requested

12. Testing and Validation

The final test run completed with 12 passing tests. Tests cover the behavior most likely to regress during provider and parser changes.

Area	Validated behavior
Schema	Exactly 20 fields; extra keys rejected; missing values use required conventions.
Parser	PDF and HTML parsing with source/page metadata.

Area	Validated behavior
Fallback	Deterministic extraction for both bid groups and corrected label handling.
Addenda	Addendum 2 due-date precedence.
Bid 2	Dell/Maryland/eMMA/model/part/delivery facts.
Chunking	Oversized text split at useful boundaries with overlap.
Merging	Known values preserved; lists deduplicated; narratives combined.
Provider normalization	Aliases and scalar lists normalized before validation.

```
python -m pytest -q
12 passed
```

13. Submission Packaging

The final archive was rebuilt after the latest parser, normalization, and retry-handler changes. It contains 37 entries and is named `emplay-rfp-extractor-submission.zip`.

Included:

- `data/input`: source PDF and HTML bid documents
- `data/output`: individual JSON files and consolidated `bid_1/bid_2` outputs
- `src`: parser, schema, extractor, configuration, and utilities
- tests: automated test suite
- `README.md`, `requirements.txt`, `.env.example`, `.gitignore`, and `main.py`

Excluded for security and cleanliness:

- `.env` and API keys
- `.venv/venv`
- `__pycache__`
- `.pyc` files
- `.pytest_cache`

14. Reproduction Instructions

From the project root on Windows PowerShell:

```
python -m venv .env
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
python main.py --input data/input --output data/output --provider none
python -m pytest -q
```

For a live provider, configure the appropriate key and model in `.env`, then pass `--provider openai`, `--provider gemini`, or `--provider groq`. A live provider should be considered successful only when the entire run completes and all generated JSON passes validation.

15. Final Status and Limitations

Item	Status
Project structure	Complete
PDF/HTML parser	Complete; pypdf-first with fallback

Item	Status
Strict 20-field schema	Complete
Provider integrations	Complete, but external availability varies
Offline fallback	Complete and used for final local run
Chunking and merging	Complete
Bid consolidation	Complete
Addendum precedence	Complete
Automated tests	12 passed
Clean ZIP submission	Rebuilt and verified
Successful full live LLM run	Not completed because providers returned quota, rate-limit, or overload errors

Honest interpretation: the submitted JSON outputs are validated deterministic fallback outputs. The code includes live LLM integrations, but no provider completed a full run reliably during final validation.

16. Conclusion

The completed project delivers a reproducible, auditable RFP extraction system rather than a one-off script. It handles heterogeneous procurement documents, enforces a stable contract, preserves evidence, combines bid documents, applies addenda, supports optional LLM providers, and remains usable when external APIs are unavailable.

Final submission artifact: employ-rfp-extractor-submission.zip

Project report: employ-rfp-extractor-project-report.pdf